SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05
NAME: Mukesh Raj L
REG NO: 192571036

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

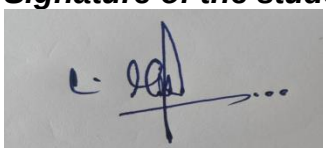
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Apply relational algebra operations (σ, π, ∪, −, ×, ⋈) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	**5**
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	**5**
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	**5**
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	**5**
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	**5**
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	**5**
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	**5**
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with	BL3 – Apply	**5**

	integrity constraint enforcement for a given scenario.		
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	**5**
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	**5**

Code of Conduct:

I Mukesh Raj L (192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with	Mostly correct steps with minor	Disorganized steps; multiple errors	No clear process
		correct approach throughout	mistakes		
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1)

Employee

EmpID	EmpName	DeptID	Salary
101	Arun	D1	50000
102	Bala	D2	60000
103	Charan	D1	55000

Department

DeptID	DeptName
D1	IT
D2	HR

1. Selection (σ)

Purpose: Select rows satisfying a condition.

Find employees with salary greater than 50,000

σSalary>50000(Employee)

Result

EmpID	Emp Name	DeptID	Salary
102	Bala	D2	60000
103	Charan	D1	55000

2. Projection (π)

Purpose: Select only required columns.

Display employee names only

πEmpName(Employee)

Result

EmpName
Arun
Bala
Charan

3. Union (∪)

Union combines tuples from two compatible relations.

Example:

IT Employees

EmpID
101
103

HR Employees

EmpID
102

ITEmployees∪HREmployees

Result

EmpID
101
102
103

4. Set Difference (−)

Returns tuples present in the first relation but not in the second.

Example:

Employees = {101,102,103}

ITEmployees = {101,103}

Employees−ITEmployees

Result

EmpID
102

5. Cartesian Product (×)

Combines every employee with every department.

$$Employee \times Department$$

If Employee has **3 records** and Department has **2 records**,

Total rows

3×2=63

Example:

EmpName	DeptName
Arun	IT
Arun	HR
Bala	IT
Bala	HR
Charan	IT
Charan	HR

6. Join (⋈)

Join combines related tuples based on a common attribute.

Employee⋈Employee.DeptID=Department.DeptID Department

Result

EmpID	EmpName	DeptName	Salary
101	Arun	IT	50000
102	Bala	HR	60000
103	Charan	IT	55000

2)

Explanation

DDL (Data Definition Language) commands are used to create and define the structure of database objects like tables.

Constraints used:

- **PRIMARY KEY** – Uniquely identifies each record.
- **NOT NULL** – Column cannot contain NULL values.
- **UNIQUE** – Prevents duplicate values.
- **FOREIGN KEY** – Maintains referential integrity between tables.

Step 1: Create DEPARTMENT Table

```
CREATE TABLE DEPARTMENT
(
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

Explanation

- DeptID → Primary Key.
- DeptName → Cannot be NULL and must be unique.

Step 2: Create STUDENT Table

```
CREATE TABLE STUDENT
(
   SID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   DOB DATE NOT NULL,
   DeptID INT,
   FOREIGN KEY (DeptID)
   REFERENCES DEPARTMENT(DeptID)
);
```

Explanation

- SID → Primary Key.
- Name → Cannot be NULL.
- DOB → Stores Date of Birth.
- DeptID → Foreign Key referencing DEPARTMENT.

Table Structure

DEPARTMENT

Column	Data Type	Constraint
DeptID	INT	PRIMARY KEY
DeptName	VARCHAR(50)	NOT NULL, UNIQUE

STUDENT

Column	Data Type	Constraint
SID	INT	PRIMARY KEY
Name	VARCHAR(50)	NOT NULL
DOB	DATE	NOT NULL
DeptID	INT	FOREIGN KEY REFERENCES DEPARTMENT(DeptID)

3)

Example Sales Table

SaleID	Product	Department	Amount
1	Laptop	Electronics	50000
2	Mouse	Electronics	1000
3	Chair	Furniture	3000
4	Table	Furniture	7000
5	Laptop	Electronics	55000

1. COUNT()

Find the number of sales in each department.

SELECT Department, COUNT(*) AS TotalSalesFROM SalesGROUP BY Department;

Output

Department	TotalSales
Electronics	3
Furniture	2

2. SUM()

Find the total sales amount for each department.

SELECT Department, SUM(Amount) AS TotalAmountFROM SalesGROUP BY Department;

Output

Department	TotalAmount
Electronics	106000
Furniture	10000

3. AVG()

Find the average sales amount in each department.

SELECT Department, AVG(Amount) AS AverageAmountFROM SalesGROUP BY Department;

Output

Department	AverageAmount
Electronics	35333.33

Department	AverageAmount
Furniture	5000

4. MAX()

Find the highest sale amount in each department.

SELECT Department, MAX(Amount) AS HighestSaleFROM SalesGROUP BY Department;

Output

Department	HighestSale
Electronics	55000
Furniture	7000

5. MIN()

Find the lowest sale amount in each department.

SELECT Department, MIN(Amount) AS LowestSaleFROM SalesGROUP BY Department;
Output

|Electronics|1000|
|Furniture|3000|

6. HAVING Clause

Display only departments whose total sales amount is greater than 50,000.

SELECT Department, SUM(Amount) AS TotalAmountFROM SalesGROUP BY DepartmentHAVING SUM(Amount) > 50000;
Output

Department	TotalAmount
Electronics	106000

5)

Example Tables
Employee

EmpID	EmpName	ProjectID

EmpID	EmpName	ProjectID
101	Arun	P1
102	Bala	P2
103	Charan	NULL
104	Deepak	P3

Project

ProjectID	ProjectName
P1	Website
P2	Mobile App
P4	AI System

1. INNER JOIN

Definition: Returns only the matching records from both tables.

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM Employee E
INNER JOIN Project P
ON E.ProjectID = P.ProjectID;Output
```

EmpID	EmpName	ProjectName
101	Arun	Website
102	Bala	Mobile App

Explanation: Only employees assigned to existing projects are displayed.

2. LEFT OUTER JOIN

Definition: Returns all records from the left table (Employee) and matching records from the right table (Project). If there is no match, NULL is returned.

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM Employee E
LEFT OUTER JOIN Project P
ON E.ProjectID = P.ProjectID;Output
```

EmpID	EmpName	ProjectName
101	Arun	Website
102	Bala	Mobile App
103	Charan	NULL

EmpID	EmpName	ProjectName
104	Deepak	NULL

Explanation: All employees are displayed. Employees without a matching project have `NULL` values.

3. RIGHT OUTER JOIN

Definition: Returns all records from the right table (Project) and matching records from the left table (Employee).

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM Employee E
RIGHT OUTER JOIN Project P
ON E.ProjectID = P.ProjectID;Output
```

EmpID	EmpName	ProjectName
101	Arun	Website
102	Bala	Mobile App
NULL	NULL	AI System

Explanation: All projects are displayed. Projects without an assigned employee have `NULL` values for employee details.

4. FULL OUTER JOIN

Definition: Returns all records from both tables. If there is no match, `NULL` values are returned.

SQL Query

```
SELECT E.EmpID, E.EmpName, P.ProjectName
FROM Employee E
FULL OUTER JOIN Project P
ON E.ProjectID = P.ProjectID;Output
```

EmpID	EmpName	ProjectName
101	Arun	Website
102	Bala	Mobile App
103	Charan	NULL
104	Deepak	NULL
NULL	NULL	AI System

Explanation: Displays every employee and every project, including unmatched records.

5. CROSS JOIN

Definition: Returns the Cartesian product of the two tables (every employee paired with every project).

SQL Query

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
CROSS JOIN Project P;
```

If there are **4 employees** and **3 projects**, the result contains:

4×3=12

Sample Output

EmpName	ProjectName
Arun	Website
Arun	Mobile App
Arun	AI System
Bala	Website
Bala	Mobile App
...	...

Explanation: Every employee is combined with every project.

6)

What is a View?

A **View** is a **virtual table** created using a `SELECT` statement. It does not store data itself but displays data from one or more tables.

Advantages of a View:

- Improves security by restricting access to specific data.
- Simplifies complex queries.
- Provides a customized view of data.

Assume the Tables

Employee

EmpID	EmpName	DeptID	Salary
101	Arun	D1	50000
102	Bala	D1	60000
103	Charan	D2	40000
104	Deepak	D2	55000

Department

DeptID	DeptName
D1	IT
D2	HR

Step 1: Create the View

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
    COUNT(E.EmpID) AS TotalEmployees,
    AVG(E.Salary) AS AverageSalary
FROM Department D
JOIN Employee E
ON D.DeptID = E.DeptID
GROUP BY D.DeptName;Explanation
```

- CREATE VIEW DeptSalaryView AS → Creates a view named **DeptSalaryView**.
- COUNT(E.EmpID) → Counts the number of employees in each department.
- AVG(E.Salary) → Calculates the average salary of employees.
- JOIN → Combines Department and Employee tables.
- GROUP BY D.DeptName → Groups records by department name.

Step 2: Query the View

```
SELECT * FROM DeptSalaryView;
```

Expected Output

DeptName	TotalEmployees	AverageSalary
IT	2	55000
HR	2	47500

Another Query on the View

Display only departments with an average salary greater than 50,000.

```
SELECT *
FROM DeptSalaryView
WHERE AverageSalary > 50000;
```

Output

DeptName	TotalEmployees	AverageSalary
IT	2	55000

Explanation

- A **View** is a virtual table based on one or more tables.
- COUNT() counts the employees in each department.
- AVG() calculates the average salary.
- JOIN combines the Department and Employee tables.
- The view can be queried like a normal table.

7)

Required Relations

Assume the following relations:

STUDENT

SID	Name
101	Arun
102	Bala
103	Charan

COURSE

CID	CourseName
C1	DBMS
C2	OS
C3	CN

ENROLLMENT

SID	CID
101	C1
101	C2
101	C3
102	C1
102	C2
103	C2
103	C3

Step 1: Project Required Attributes

Get only the student-course pairs.

πSID,CID(ENROLLMENT)

Get all course IDs.

πCID(COURSE)

Step 2: Use Division (÷)

The **Division (÷)** operator is used to find entities related to **all** values in another relation.

πSID,CID(ENROLLMENT)$\div\pi$CID(COURSE)

This returns the Student IDs of students enrolled in every course.

Step 3: Join with STUDENT

To display student names, join the result with the STUDENT table.

$\pi Name(STUDENT \bowtie (\pi SID,CID(ENROLLMENT) \div \pi CID(COURSE)))$

Explanation

- $\pi SID,CID(ENROLLMENT)$ → Retrieves student-course pairs.
- $\pi CID(COURSE)$ → Retrieves all course IDs.
- ÷ (Division) → Finds students enrolled in all courses.
- ⋈ (Join) → Combines the result with the STUDENT table.
- $\pi Name$ → Displays only the student names.

8)

What is DML?

DML (Data Manipulation Language) is used to manipulate data in database tables.

The main DML commands are:

- **INSERT** – Adds new records.
- **UPDATE** – Modifies existing records.
- **DELETE** – Removes records.

1. INSERT Statement

Insert a new student into the STUDENT table.

```
INSERT INTO STUDENT (SID, Name, DOB, DeptID)
VALUES (101, 'Arun', '2005-05-15', 1);
```

Explanation

- Inserts a new student record.
- DeptID = 1 must already exist in the DEPARTMENT table (Foreign Key constraint).
- SID must be unique (Primary Key constraint).

2. UPDATE Statement

Update the department of student with SID = 101.

```
UPDATE STUDENT
SET DeptID = 2
WHERE SID = 101;
```

Explanation

- Changes the student's department to 2.
- DeptID = 2 must exist in the DEPARTMENT table; otherwise, the update is rejected due to the Foreign Key constraint.

3. DELETE Statement

Delete the student whose SID is 101.

```
DELETE FROM STUDENT
WHERE SID = 101;
```

Explanation

- Deletes the student record with SID = 101.
- If the record is referenced by another table through a Foreign Key, deletion may be restricted unless cascading rules are defined.

Integrity Constraint Enforcement

The database enforces the following constraints:

- **PRIMARY KEY**
  - SID must be unique.
  - Duplicate SID values are not allowed.
- **FOREIGN KEY**
  - DeptID in STUDENT must exist in the DEPARTMENT table.
- **NOT NULL**
  - Mandatory fields such as Name and DOB cannot be left empty.

Example of Constraint Violation

This statement fails because DeptID = 10 does not exist in the DEPARTMENT table.

```
INSERT INTO STUDENT (SID, Name, DOB, DeptID)

VALUES (102, 'Bala', '2005-03-20', 10);
```

Reason: Foreign Key constraint violation.

Integrity Constraints

- **PRIMARY KEY** – Prevents duplicate SID values.
- **FOREIGN KEY** – Ensures DeptID exists in the DEPARTMENT table.
- **NOT NULL** – Prevents mandatory fields from being left empty.

9)

What is Tuple Relational Calculus (TRC)?

Tuple Relational Calculus (TRC) is a non-procedural query language. It describes what data is required, not how to retrieve it.

The general form of TRC is:

{ t| Condition(t) }

where:

- **t** = Tuple (row)
- **|** = "such that"
- **Condition** = Criteria that the tuple must satisfy

Assume the Relations

Employee

EmpID	EmpName	DeptID	Salary
101	Arun	D1	55000
102	Bala	D2	45000
103	Charan	D1	60000

Department

DeptID	DeptName
D1	IT
D2	HR

Tuple Relational Calculus Expression

```
{ E | Employee(E) AND E.Salary > 50000 AND
  EXISTS D ( Department(D) AND
          D.DeptID = E.DeptID AND
          D.DeptName = 'IT' ) }
```

Explanation:
1. Employee(E) specifies that E is a tuple in the Employee relation.
2. E.Salary > 50000 selects employees earning more than 50,000.
3. EXISTS D indicates that there exists a tuple D in the Department relation.
4. D.DeptID = E.DeptID matches the employee with their department.
5. D.DeptName = 'IT' ensures that the employee works in the IT department.

Expected Output

EmpID	EmpName	DeptID	Salary
101	Arun	D1	55000
103	Charan	D1	60000

10)

Airline Reservation System - Relational Schema

1. PASSENGER Table

```
PASSENGER(
   PassengerID INT PRIMARY KEY,
```

```
    PassengerName VARCHAR(50),
    Age INT,
    Gender VARCHAR(10)
);
```

2. FLIGHT Table

```
FLIGHT(
    FlightID INT PRIMARY KEY,
    FlightName VARCHAR(50),
    Source VARCHAR(50),
    Destination VARCHAR(50)
);
```

3. BOOKING Table

```
BOOKING(
    BookingID INT PRIMARY KEY,
    PassengerID INT,
    FlightID INT,
    JourneyDate DATE,
    SeatNo VARCHAR(10),
    FOREIGN KEY (PassengerID) REFERENCES PASSENGER(PassengerID),
    FOREIGN KEY (FlightID) REFERENCES FLIGHT(FlightID)
);
```

SQL Queries

Query 1: INNER JOIN

Display passenger names and their flight details.

```
SELECT P.PassengerName, F.FlightName, F.Source, F.Destination
FROM PASSENGER P
INNER JOIN BOOKING B
ON P.PassengerID = B.PassengerID
INNER JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

Query 2: LEFT JOIN

Display all passengers along with their booking details.

```
SELECT P.PassengerName, B.BookingID
FROM PASSENGER P
LEFT JOIN BOOKING B
ON P.PassengerID = B.PassengerID;
```

Query 3: Subquery

Find passengers who booked Flight ID = 101.

```
SELECT PassengerName
FROM PASSENGER
WHERE PassengerID IN
```

```
(
   SELECT PassengerID
   FROM BOOKING
   WHERE FlightID = 101
);
```

Query 4: Create View

Create a view showing passenger and flight details.

```
CREATE VIEW FlightBookingView AS
SELECT P.PassengerName,
      F.FlightName,
      F.Source,
      F.Destination
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

Query 5: Query on the View

```
SELECT * FROM FlightBookingView;
```

Explanation

1. PASSENGER table stores passenger details.
2. FLIGHT table stores flight information.
3. BOOKING table connects passengers and flights using foreign keys.
4. INNER JOIN displays passengers with their booked flights.
5. LEFT JOIN displays all passengers, even those without bookings.
6. Subquery finds passengers booked on a specific flight.
7. VIEW stores a virtual table combining passenger and flight details.